PES University, Department of Computer Science and Engineering

SE Lab 1: Use-Case Flow

Problem Statement 44: Database Query Performance Profiler

Name: R Saurav Srrinivas SRN: PES1UG24CS352 Deliverable 3

Use Case ID	UC-03
Use Case Name	Recommend Index Candidates
Primary Actor	Database Administrator
Supporting Actor	Monitored Database Server (Postgres / MySQL)
Related Requirements	FR-001, FR-002, FR-005, NFR-002
Relationships	Includes UC-02 Analyse SQL Execution Plan. Extended by UC-07 Simulate Index on Replica.
Trigger	The DBA asks for index suggestions on a slow query, or the scheduled analysis job runs after a collection cycle.

Preconditions

1. The DBA is logged in and has the DBA role, as required by NFR-002.
2. At least one database has been registered, is reachable, and has profiling switched on (FR-003).
3. At least one slow query fingerprint has been collected for that database in the selected time window (FR-002).

Postconditions

Success: A ranked list of index suggestions is saved against the fingerprint. Each entry holds the suggested index, the current plan cost, the estimated cost after the index and the expected saving. The list is then available to the weekly digest and to triage (FR-005). Nothing on the monitored database is changed.

Failure: Nothing is saved. The fingerprint is marked as failed with a reason, the failure is written to the audit log, and again nothing on the monitored database is changed.

Main Success Scenario

1. The DBA picks a database and then a slow query fingerprint from the ranked list.
2. The system shows the fingerprint summary: the normalised SQL, how many times it ran, the total time and the average time.
3. The DBA clicks "Recommend Indexes".
4. The system fetches the current plan for that query from the monitored database by running EXPLAIN. The query itself is never executed.
5. Includes UC-02, Analyse SQL Execution Plan. The system reads the plan tree, marks every sequential scan on a table that is above the large table limit, and notes the filter, join and sort columns used at each of those points.
6. The system reads the column statistics for those tables from the database catalog, mainly the number of distinct values and the fraction of nulls.
7. From this the system builds candidate indexes. Columns of a composite index are ordered most selective first, and the index type (B-tree, hash or GIN) is picked from the operators used in the query.
8. For each candidate the system estimates what the plan would cost with that index in place, and works out the saving against the current cost.
9. Candidates that save less than 10 percent, or that repeat an index which already exists, are dropped. The rest are ranked by saving.
10. The system saves the ranked list against the fingerprint with the time of analysis.
11. The system displays each suggestion with a ready CREATE INDEX statement, the expected saving and a rough index size.
12. The DBA marks each suggestion as Accepted, Rejected or Deferred. The use case ends here.

At step 11 the DBA may also choose to run UC-07, Simulate Index on Replica, to try a candidate on the staging replica first. This is the extend relationship shown in the diagram.

Alternate Flow 4a: The Execution Plan Cannot Be Fetched

4a1. The plan cannot be fetched, either because the database is unreachable, or the EXPLAIN times out after 10 seconds, or the profiler account has no permission on one of the tables.

4a2. The system stops the analysis. No statement is run and no lock is taken on the monitored database.

4a3. It retries twice, waiting 5 seconds and then 15 seconds.

4a4. If a retry works, the flow goes back to step 5 above.

4a5. If all retries fail, the fingerprint is marked as failed with the reason (unreachable, timeout or permission denied). The DBA is shown a message naming the host or the missing privilege, an audit entry is written, and the use case ends at the failure postcondition.

Alternate Flow 9a: No Useful Index Found

9a1. Every candidate gets dropped, either because the saving is under 10 percent or because a similar index already exists.

9a2. The system records a "no suggestion" result, tells the DBA that an index is unlikely to help here, and points to query rewriting or partitioning instead. The use case ends.